Training-Free Rank-One Prompt Distillation for a Quantized Gemma 4 Coding Agent

Controlled evidence, preserved failures, and limits

[**YOU:** your name]
[**YOU:** affiliation / "independent researcher"]

[**YOU:** submission date]

GUIDE. Words count *everything* (abstract, captions, tables' text, references excluded by most graders but do not rely on it). Current draft is about 1,900 words, so you have room to add about 1,000. Spend them on (1) why this question matters to an agent builder, (2) one worked example of a trajectory (see Section 4.5), (3) a figure. Do not add claims. Every sentence below is traceable to a file listed in `docs/paper-latex/SOURCES.md`.

Abstract

GUIDE. An abstract has four jobs, in this order: problem, what you did, what you found (with numbers), what the limit is. Keep one result number from each of the three regimes (greedy, sampled, competition setting). Never write "robust" or "state of the art". Judges reward calibrated honesty here because the result is mixed.

Small coding agents often fail at one step of the repair loop. We ask whether a closed-form, rank-one weight edit can carry one such behaviour, rereading source after a failed partial-line edit, into the competition's quantized Gemma 4 E4B through its standard PEFT-adapter path. The edit uses the output shift between a prompted and an unprompted agent at one decision token, whitened by background key statistics, and is installed as one rank-1 component of a rank-8 adapter on one attention output projection. Under frozen, pre-registered protocols, a static-prototype edit and the same activation-derived edit at strength 1 both failed. With strength chosen from design cases only, the edit completed 21 and 23 of 24 target workflows in two greedy runs on fresh cases, against 4–5 for the unchanged model and 0 for its sign-flipped copy; a gradient-trained rank-8 LoRA and the instruction prompt each completed 24. Under temperature-0.7 sampling one seed passed, one failed (the edit produced empty completions) and one was incomplete. In the competition's own setting (temperature 0.2, thinking enabled), one batch passed (15 of 24 versus 3 for base) and one did not (10 versus 6). The method is prior work in model editing; our contribution is a controlled deployment study with failures preserved, on one synthetic behaviour and no real repairs.

1 Introduction

GUIDE. Write this section in your own voice; it is where a reader decides whether you are a researcher or a prompt-engineer. Structure: (a) the practical problem (context cost and brittleness of system prompts on small models), (b) the idea from the weight-space-programming article [1] in one sentence, (c) the narrow, checkable question we actually test, (d) a bullet list of contributions that matches Section 2 exactly, (e) a one-sentence preview of the honest result. Cut anything that sounds like a promise.

System-prompt instructions cost context and are brittle for small models. Weight-space programming proposes that a general-purpose model can read and rewrite weights directly [4]. We test a narrower, checkable question under real deployment limits: the approved quantized E4B checkpoint, the harness's standard PEFT adapters, and vLLM serving. A design that only moves token statistics is not enough. We require a change in a specific agent behaviour that survives that path, against controls that rule out nonspecific perturbation, and we pre-register every gate.

Contributions. (i) A design-set-selected, covariance-whitened rank-one edit built from prompt-versus-base activations and deployed as a standard adapter on a quantized E4B. (ii) Controls: zero adapter, norm-matched random adapter, sign-flipped edit, and a gradient-trained LoRA baseline. (iii) A preserved record of failures, including a sampling-regime failure that greedy evaluation hid. [**YOU:** rephrase; keep exactly these three claims]

2 Related work and claim boundary

GUIDE. This section decides your novelty score. Do it carefully: for each cited work say (1) what it does, (2) how ours differs *in one clause*. Before submission, read the full texts of the three references marked † (only abstracts were checked). If any already covers a contribution, delete that contribution from the list above. Re-run the novelty search (see `docs/research/novelty.md`) on 10 November.

The algebra is established. ROME [2] applies closed-form rank-one key–value edits. Steer2Edit† [3] converts steering vectors into component-level rank-1 weight edits without training. Mazzawi et al.† [1] turn a prompt's influence into token-independent weight updates. Yin et al.† [5] compile an activation-steering direction into a persistent closed-form edit under a null-space constraint, for backdoor injection. Gradient-trained prompt distillation into a LoRA is the standard alternative. We claim no new editing method; our contribution is empirical. We do not test blind weight reading or the other claims of [4].

3 Method

GUIDE. Reproducibility test: could a stranger rebuild the adapter from this section plus the repository? Keep the formula, the layer choice, the strength rule and the exact claim "no evaluation case enters design or selection". Add one sentence on why layer 29 and one on why whitening matters (it removes the dominant background directions so the edit fires on the target key only).

3.1 Target behaviour

After an edit leaves a syntax error, the base agent repeats a stale replacement; a prompt saying "reread, then replace the whole line" makes it call `read_file` first. At the decision token, 12 of 12 design cases reread under that prompt and 0 of 12 under the base prompt. Cases are seeded synthetic tool sessions (an arithmetic `adjust` function) checked by a restricted interpreter and the official editor; a workflow is complete only if the final source is correct, was tested after the last edit, and was submitted.

3.2 Construction

On 12 design cases we replay each history under both prompts and record, at the last token, the input k and output o of `self_attn.o_proj` in layers 12–38. Let d be the mean prompted-minus-base

output and k^* the mean base input; the layer maximizing $\|d\|/\|\bar{o}_{\text{base}}\|$ is used (layer 29, input width 4096). A ridge-regularized covariance C of that input over all tokens of 12 non-target design cases whitens the key:

$$\Delta W \;=\; \alpha\,\frac{d\,(C^{-1}k^*)^\top}{k^{*\top}C^{-1}k^*}, \qquad \Delta W\,k^* = \alpha\,d. \tag{1}$$

It is installed as one rank-1 component of a rank-8, $\alpha_{\text{LoRA}} = 8$ bf16 adapter; the realized shift error is 0.17%. The strength α is the smallest value in $\{2, 3, 4, 6, 8\}$ for which greedy decoding on design cases rereads in at least 11 of 12 target cases and leaves the first tool call unchanged in all 12 non-target cases ($\alpha = 4$). No evaluation case enters design or selection.

3.3 Controls and baseline

Base; zero adapter (same A, $B = 0$); seeded random adapter with matched factor norms; sign-flipped edit; the instruction prompt; and a base repeat (v2 only). The gradient baseline is a rank-8 LoRA on the same module trained on the same 24 design cases (teacher: the prompted agent's continuation; preservation targets on non-target cases), stopped at the first checkpoint meeting the same design-set rule (step 40). Adapter application in vLLM was verified separately: active adapters shift first-token log-probabilities by 0.09–0.76 nats on three fixed prefixes with zero control drift.

3.4 Protocols

Each experiment was frozen, with cases, gates and budget hashed in a ledger before any inference. Evaluation cases are seeded, disjoint from every earlier coefficient pair, and never used for design.

4 Results

GUIDE. Lead with the failures (already done below): it is the most credible thing in the paper. Make one figure (Figure 1) your anchor. If you add anything, add a short worked trajectory (Section 4.5) taken from a real run row in `artifacts/wsp-v4/*/run.json`; quote the first tool call of base vs. edit. Report p-values only where the protocol pre-registered a test.

4.1 Failures first

A static-prototype edit built from token embeddings completed 9 of 12 workflows versus 10 for base and 12 for the prompt. The activation-derived edit at $\alpha = 1$ (36 cases, 8 arms, 288 trajectories) completed 3 of 24 target workflows versus 4 for base, with no rereads, although it was applied. A post-hoc diagnostic on those evaluation prompts showed the key fires (dot product 0.99) and the log-probability margin grows monotonically with α, but $\alpha = 1$ recovers about 6% of the prompt's margin. That diagnostic motivated the strength grid and is a disclosed researcher degree of freedom.

4.2 Greedy decoding, fresh cases ($\alpha = 4$)

Run 1 (216 trajectories) passed all four pre-registered gates: the edit beat base, zero and random by net +16, +16, +17 target workflows (one-sided exact sign tests, $p < 10^{-4}$), beat the sign-flipped edit by +21 and lost no non-target case. Run 2 (180 trajectories) repeated the pattern. Identical adapters and settings gave 21 and 23, so single greedy draws carry about ±2 serving noise. The sign-flipped edit removed even the base's occasional successes, so the gain is directional. Under the frozen comparability rule (within 3 workflows) the edit is comparable to the gradient-trained LoRA

Table 1: Target workflows complete, of 24 (greedy). Non-target workflows were 12 of 12 in every arm. A dash means the arm was not run.

Arm	Run 1	Run 2
base	5	4
zero adapter	5	–
random, norm-matched	4	–
edit	**21**	**23**
sign-flipped edit	0	0
gradient-trained LoRA	–	24
prompt	24	24

(23 vs. 24), which trained in about five minutes; the edit's value is that it needs no optimizer, not accuracy.

4.3 Sampling at temperature 0.7

Seed 101: edit 24, base 18, sign-flipped 6, prompt 24 (pass). Seed 103: edit 4, base 24, sign-flipped 0, prompt 24 (fail); in 20 of 24 target cases the edit's first sampled completion was empty (one token, then stop). Seed 102 was cancelled at 63 of 144 rows and is incomplete. Robustness is not established. Because every request in a batch shared one sampling seed, each seed is close to one draw per arm; within-seed counts are not independent samples.

4.4 Competition setting: temperature 0.2 with thinking

We then froze a further protocol: 36 new cases, temperature 0.2, top_p 0.95, thinking enabled, independent per-trajectory seeds, two batches (seeds 201, 202), and the same $\alpha = 4$ edit with no re-tuning. Batch A passed every gate (edit beat base by net +12, $p = 0.0002$, and the sign-flipped

Table 2: Target workflows complete, of 24, in the competition setting.

Batch	base	edit	sign-flipped	prompt
A (seed 201)	3	15	2	10
B (seed 202)	6	10	4	11

edit by +13; one non-target loss). Batch B failed the net-+6 gate (edit beat base by +4, $p = 0.17$). By the frozen rule the outcome is mixed. The first action was `read_file` in 12 and 10 of 24 edit trajectories, against 6 and 5 for base and 18 and 21 for the prompt: the edit moves behaviour in the intended direction at roughly half the prompt's strength. The empty-completion failure did not recur. Between 8 and 15 of 24 trajectories in every arm, the prompt included, ended on the six-request budget, which caps completions in thinking mode.

4.5 Worked example

GUIDE. Write this yourself (about 150 words). Pick one target case from batch A where base edits blindly and the edit rereads first. Show: the file state, base's first call, the edit arm's first call, and the final outcome. Rows are in `artifacts/wsp-v4/20261108T080127Z-wsp-v4-a/run.json` (events with stage `case`). If you cannot find a clean example, delete this subsection.

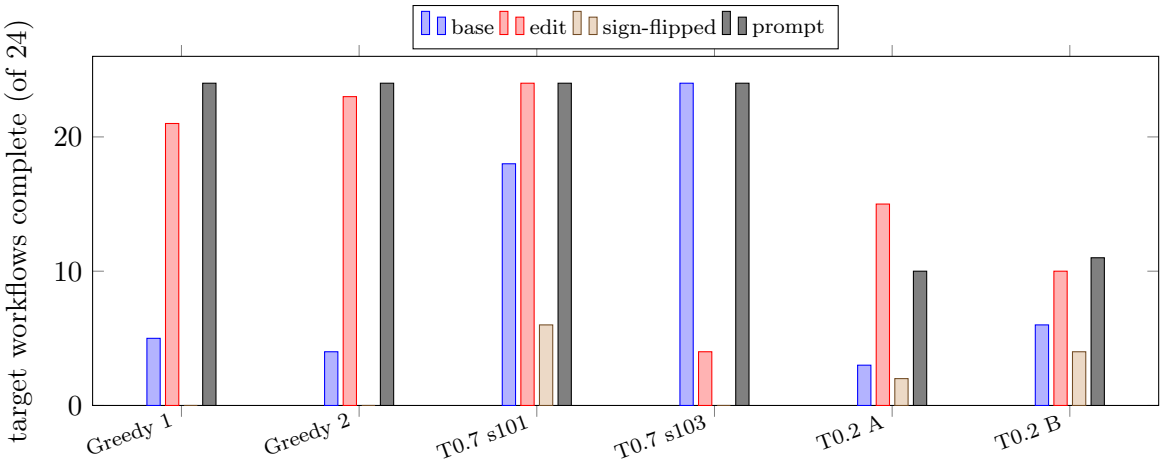


Figure 1: Every regime in one view: the edit's advantage over base is clear under greedy decoding, reverses in one sampled seed, and is partial at the competition's setting. (Seed 102 omitted: incomplete.)

[**YOU:** worked example]

5 Limitations

GUIDE. Do not shorten this section. State each limit as a fact, not an apology. A reader who finds a limit you hid will discount everything else.

One synthetic behaviour in one template family; fresh cases differ from design cases only in coefficients. No real-repair claim: the edit has not been run on real issues, and earlier real-task screens of prompt changes showed no repair gain. In v4 the 4,096-token thinking budget was approximated by enabling thinking with an 8,192-token output limit and was not separately enforced, and a six-request cap truncates many thinking-mode trajectories in every arm. Non-target cases are at ceiling (every arm, including the harmful sign-flipped edit, completes 12 of 12), so side-effect detection is weak. Greedy single draws vary by about ±2 workflows. The gradient baseline used untuned hyperparameters. We did not ablate numerical weight access, so we cannot say whether seeing weights helped design. The edit was designed by an LLM assistant that knew prior traces. The strength grid was motivated by a post-hoc diagnostic.

6 Reproducibility

Protocols, case generators, adapter builders, ledger and hashed reports are in the repository. Runs used Modal A10G with the competition's pinned wheels and `gemma-4-e4b-it-qat-w4a16-ct` v2, for about USD 11 in total. Incidents are recorded: a lost terminal event in the first panel (gate recomputed from streamed rows with unit-tested code), a batch cut by its execution limit, a report-path collision that forced a seed rerun, and a cancelled batch. [**YOU:** decide: link public code? yes/no]

7 Conclusion

GUIDE. Three sentences: what holds (directional effect, comparable to a trained LoRA when greedy), what does not (reliable under sampling, beating the prompt), what the next experiment would be (real repair issues, a second behaviour, an enforced thinking budget). No new claims.

[**YOU:** write the conclusion in your own words]

References

[1] Mazzawi, Dherin, Munn, Goldwaser, Wunder, and Gonzalvo. Transmuting prompts into weights. *arXiv preprint arXiv:2510.08734*, 2025. Verify full author names from the paper.

[2] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. *arXiv preprint arXiv:2202.05262*, 2022.

[3] Sun, Yan, Wang, and Weng. Steer2Edit: From activation steering to component-level editing. *arXiv preprint arXiv:2602.09870*, 2026. Verify full author names from the paper.

[4] Takehi et al. Sparks of weight-space programming. `https://rikiyat.github.io/blog/weight-space-programming`, 2026. 2 November 2026. Verify author list before submission.

[5] Yin et al. Compiling activation steering into weights via null-space constraints for stealthy backdoors. In *Proceedings of ACL*, pages 26228–26245, 2026. Verify author list.